

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. At least six superkeys

- {EmpID}
- {SSN}
- {Email}
- {EmpID, SSN}
- {EmpID, Name}
- {SSN, Email, Phone}
- {EmpID, Department, Salary}
- {EmpID, SSN, Email, Phone, Name, Department, Salary} (the full set of attributes)

Any superset of a candidate key is itself a superkey, so once {EmpID}, {SSN}, and {Email} are known to be candidate keys, adding any other attribute to one of them still produces a valid (non-minimal) superkey.

2. Candidate keys

{EmpID}, {SSN}, and {Email} are each candidate keys — every value in the sample data is unique for these three columns, and each is a minimal set (removing the single attribute leaves no unique identifier). Phone and Name are not candidate keys because real organizations allow shared desk/office phone numbers and duplicate names.

3. Choice of primary key

EmpID is the best choice for primary key. Reasoning:

- Stability: EmpID is an internally-generated surrogate value that never changes over an employee's tenure, whereas Email addresses are frequently reissued (e.g., after a name change or a company domain migration) and SSN is normally treated as immutable but is not always available (e.g., contractors, international hires) at the time a record is created.
- Privacy/compliance: SSN is sensitive personally identifiable information. Using it as a primary key means it propagates into every child table as a foreign key, multiplying exposure risk. Best practice is to store SSN as a regular (unique) attribute, not the key.
- Domain independence: EmpID is not tied to any external system or format, so it will never need to change if company email conventions or government ID formats change.

4. Can two employees share the same phone number?

Yes. The sample data happens to show three distinct numbers, but nothing in the relation's semantics guarantees uniqueness. Phone numbers are commonly shared — e.g., two employees sitting at the same desk, sharing a departmental extension, or one being a temporary forwarding number for the other during onboarding. Because uniqueness cannot be guaranteed by the business rules, Phone cannot be treated as a candidate key even though the current sample data is coincidentally unique.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum attributes needed for the primary key

Primary key = {StudentID, CourseCode, Section, Semester, Year}

2. Why each attribute is necessary

- StudentID — distinguishes which student the registration row belongs to; without it, rows for different students in the same course/section/term would collide.

- CourseCode — a student may be registered in several different courses in the same semester, so the course must be part of the key.
- Section — the business rule only forbids re-registering in the same section in the same semester, implying a student could conceivably be linked to more than one section in edge cases (e.g., lab + lecture sections); Section distinguishes which specific offering is meant.
- Semester — a student can retake the same course in a different semester, so Semester must be part of the key to allow that new row to coexist with the old one.
- Year — semester names (Fall, Spring) repeat every year, so Year is required to distinguish, e.g., Fall 2024 from Fall 2025.

3. Additional candidate keys

None. Grade and Credits are both descriptive, dependent attributes (Credits is really a property of the course/section, not an identifier, and Grade is assigned after the fact) — neither can uniquely identify a registration on its own or in combination with a subset of the key attributes above.

Task 1.2: Foreign Key Design

Given tables:

```
Student(StudentID, Name, Email, Major, AdvisorID)
Professor(ProfID, Name, Department, Salary)
Course(CourseID, Title, Credits, DepartmentCode)
Department(DeptCode, DeptName, Budget, ChairID)
Enrollment(StudentID, CourseID, Semester, Grade)
```

Foreign key relationships

Table.Column (FK)	References	Business meaning
Student.AdvisorID	Professor.ProfID	Each student is assigned one faculty advisor.
Course.DepartmentCode	Department.DeptCode	Each course is offered by exactly one department.
Department.ChairID	Professor.ProfID	Each department is chaired by one professor.
Enrollment.StudentID	Student.StudentID	Each enrollment row belongs to one student.
Enrollment.CourseID	Course.CourseID	Each enrollment row is for one course.

Note the circular reference between Professor and Department (a professor can advise students and belong informally to a department via Student.Major, and a department has a ChairID pointing to a Professor). This is a normal and valid pattern as long as one of the two tables is created first without the FK constraint enabled, or the constraint is added after both tables exist.

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System

1. Entities (strong vs. weak)

Entity	Type	Reasoning
Patient	Strong	Has its own key, PatientID, and exists independently.
Doctor	Strong	Has its own key, DoctorID.
Department	Strong	Has its own key, DeptCode.
Appointment	Strong	Has its own key, AppointmentID; existence depends logically on a patient and doctor but it is identified by its own key, so it is modeled as strong.
Prescription	Weak	Has only a partial key, PrescriptionID, and cannot be uniquely identified without also knowing the Doctor (Writes) / Patient (Receives) that owns it.
Room	Weak	RoomNumber alone is not unique — 'room 101 in Cardiology is different from room 101 in Neurology' — so Room is only unique once combined with its owning Department via the identifying relationship Has.

2. Attributes (classified)

Entity	Attributes	Classification
Patient	PatientID	Simple, key
	Name	Simple
	BirthDate	Simple
	Address (Street, City, State, Zip)	Composite
	PhoneNumbers	Multi-valued
Doctor	InsuranceInfo	Composite (provider + policy number)
	DoctorID	Simple, key
	Name	Simple
	Specializations	Multi-valued
	PhoneNumber	Simple
Department	OfficeLocation	Simple
	DeptCode	Simple, key
	DeptName	Simple
Appointment	Location	Simple
	AppointmentID	Simple, key
	DateTime	Simple
	Purpose	Simple
Prescription	Notes	Simple
	PrescriptionID	Simple, partial key (discriminant)
	Dosage	Simple
Room	Instructions	Simple
	RoomNumber	Simple, partial key (discriminant)

3. Relationships and cardinalities

Relationship	Participants	Cardinality	Participation
Attends	Doctor — Appointment	1 : N	Doctor: partial, Appointment: total (every appointment has exactly one attending doctor)
Schedules	Patient — Appointment	1 : N	Appointment: total (every appointment is for exactly one patient)
Writes	Doctor — Prescription	1 : N (identifying)	Prescription: total

Receives	Patient — Prescription	1 : N (identifying)	Prescription: total
Works In	Doctor — Department	N : 1	Doctor: total
Has	Department — Room	1 : N (identifying)	Room: total

Prescription is doubly-owned: its full key is (DoctorID, PatientID, PrescriptionID) — the identifying owners contribute the composite portion, and PrescriptionID is the discriminant that distinguishes multiple prescriptions written by the same doctor to the same patient.

4. ER Diagram

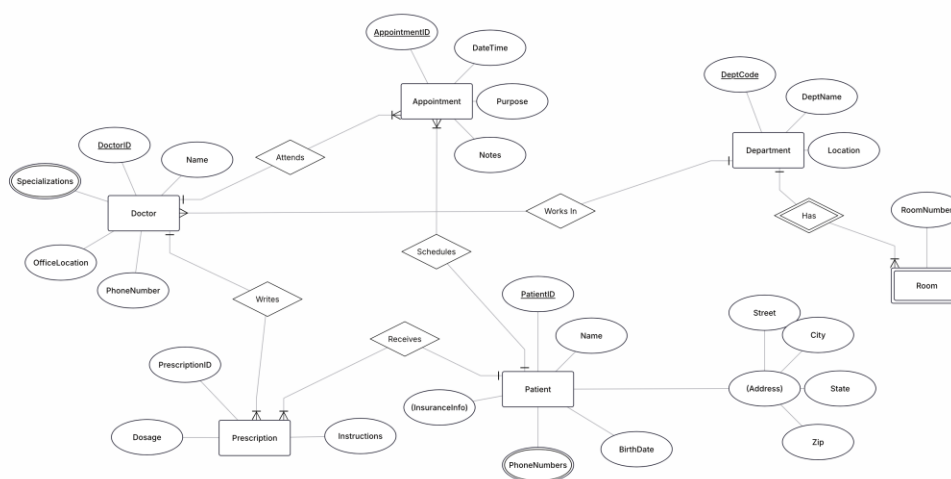


Figure 1 — Hospital Management System ER diagram (ERDPlus)

5. Primary keys

Marked (underlined) in the diagram above: PatientID, DoctorID, DeptCode, AppointmentID. Prescription's key is the composite (owner keys + PrescriptionID); Room's key is the composite (DeptCode + RoomNumber).

Task 2.2: E-commerce Platform

1–3. ER diagram, weak entity, and M:N relationship with attributes

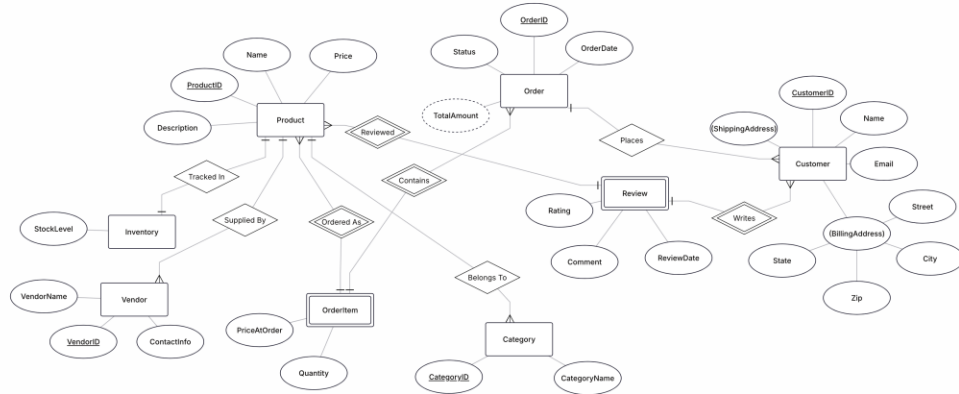


Figure 2 — E-commerce Platform ER diagram (ERDPlus)

Weak entity: OrderItem

OrderItem has no attribute that could serve as a stand-alone identifier — PriceAtOrder and Quantity are descriptive, not identifying. It exists only in the context of a specific Order (the identifying relationship Contains) combined with the Product it refers to (Ordered As), so its key is the composite (OrderID, ProductID). This is why it is drawn with a double-bordered box and double-diamond identifying relationships in the diagram.

Review is also modeled as a weak entity for the same reason: a review has no independent key of its own — it is only meaningful once tied to the Customer who wrote it (Writes) and the Product being reviewed (Reviewed).

Many-to-many relationship requiring attributes: Order ↔ Product

A single Order can contain many Products, and a single Product can appear on many Orders — a classic M:N relationship. It cannot be represented as a simple diamond because it needs its own descriptive attributes: Quantity (how many units of the product were ordered) and PriceAtOrder (the price at the time of purchase, which must be frozen even if the Product's current Price later changes). These attributes are captured by resolving the M:N relationship into the associative/weak entity OrderItem, connected to Order via Contains and to Product via Ordered As.

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional dependencies

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID
- SupervisorID → SupervisorName, SupervisorDept
- {StudentID, ProjectID} → Role, HoursWorked, StartDate, EndDate

Primary key of the original table: {StudentID, ProjectID} — a student can work on multiple projects, and a project can have multiple students, so only the pair uniquely identifies a row (specifically, one student's assignment to one project).

2. Problems

Redundancy: StudentName and StudentMajor repeat once for every project a student is assigned to. ProjectTitle, ProjectType, SupervisorID/Name/Dept repeat once for every student working on that project. SupervisorName and SupervisorDept repeat once for every project that supervisor oversees.

Anomalies:

- Update anomaly — if a student changes majors, every row containing that StudentID must be updated; missing even one row leaves the table inconsistent (two different majors listed for the same student).
- Insertion anomaly — a new project cannot be recorded until at least one student has been assigned to it, because StudentID is part of the primary key and cannot be null.
- Deletion anomaly — if the last remaining student assigned to a project is removed from the table, all information about that project (ProjectTitle, ProjectType, SupervisorID, etc.) is lost along with it.

3. 1NF

No 1NF violations are present — every attribute listed holds a single, atomic value (there are no repeating groups or multi-valued columns), so the table already satisfies 1NF.

4. 2NF

Primary key: {StudentID, ProjectID}. Partial dependencies (attributes depending on only part of the key):

- StudentID → StudentName, StudentMajor (partial — depends only on StudentID)
- ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept (partial — depends only on ProjectID)

2NF decomposition:

Student(StudentID, StudentName, StudentMajor)	PK: StudentID
Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)	PK: ProjectID
Assignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)	PK: (StudentID, ProjectID)

5. 3NF

Transitive dependency: inside the Project table, ProjectID → SupervisorID → SupervisorName, SupervisorDept. SupervisorName and SupervisorDept depend on SupervisorID, which is itself only a non-key attribute of Project — a transitive dependency on the key.

Final 3NF decomposition:

Student(StudentID, StudentName, StudentMajor)	PK: StudentID
---	---------------

Supervisor(SupervisorID, SupervisorName, SupervisorDept)	PK: SupervisorID
Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)	PK: ProjectID FK: SupervisorID -> Supervisor
Assignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)	PK: (StudentID, ProjectID) FK: StudentID -> Student FK: ProjectID -> Project

Task 4.2: Advanced Normalization

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. Primary key

{StudentID, CourseID}. Even though CourseID alone determines the instructor, time, and room (i.e., CourseID here represents one specific scheduled offering/section), a single row records one student's enrollment in one offering — and a student enrolls in many offerings, and an offering has many students — so the composite pair is the smallest set that uniquely identifies a row.

2. Functional dependencies

- StudentID → StudentMajor
- CourseID → CourseName, InstructorID, TimeSlot, Room
- InstructorID → InstructorName
- Room → Building (rooms are unique across campus, so the room alone fixes the building)

3. BCNF check

For BCNF, every determinant of a nontrivial FD must be a superkey. None of the determinants above (StudentID, CourseID, InstructorID, Room) is a superkey of {StudentID, CourseID} on its own, so the table violates BCNF on all four dependencies.

4. BCNF decomposition

1. Split off StudentID → StudentMajor: Student(StudentID, StudentMajor), PK StudentID.
2. Split off CourseID → CourseName, InstructorID, TimeSlot, Room: CourseOffering(CourseID, CourseName, InstructorID, TimeSlot, Room), PK CourseID. What remains is Enrollment(StudentID, CourseID), PK (StudentID, CourseID) — already all-key, trivially in BCNF.
3. Split off InstructorID → InstructorName from CourseOffering: Instructor(InstructorID, InstructorName), PK InstructorID.
4. Split off Room → Building from CourseOffering: RoomInfo(Room, Building), PK Room.

Final BCNF schema:

Student(StudentID, StudentMajor)	PK: StudentID
Instructor(InstructorID, InstructorName)	PK: InstructorID
RoomInfo(Room, Building)	PK: Room
CourseOffering(CourseID, CourseName, InstructorID, TimeSlot, Room)	PK: CourseID FK: InstructorID -> Instructor FK: Room -> RoomInfo
Enrollment(StudentID, CourseID)	PK: (StudentID, CourseID) FK: StudentID -> Student FK: CourseID -> CourseOffering

Check: in every resulting table the only determinant is now the whole primary key (or a foreign key referencing another table's key), so all five tables satisfy BCNF.

5. Potential information loss

The decomposition is lossless — each split was performed on a determinant that becomes the primary key of the new table, which guarantees a lossless join back to the original data (the classic $R = R_1 \bowtie R_2$ lossless-join test is satisfied because the common attribute is a key in one of the two resulting relations at every step). No FDs from the original table are lost either, so the decomposition is both lossless-join and dependency-preserving. The only practical caveat is that referential integrity (foreign key constraints) must now be enforced explicitly by the DBMS — e.g., preventing a CourseOffering row from referencing a Room value that doesn't exist in RoomInfo — since that constraint is no longer expressed implicitly by having everything in a single table.

Part 5: Design Challenge

Task 5.1: Student Clubs & Organizations System

1. ER Diagram

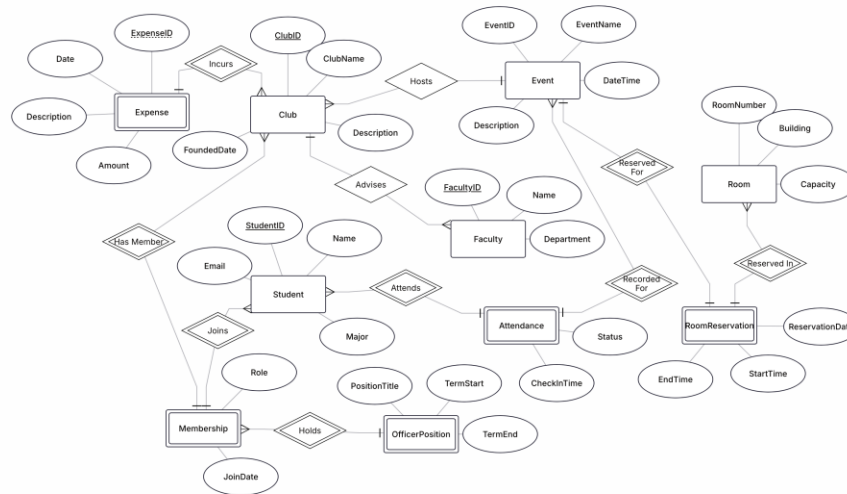


Figure 3 — Student Clubs & Organizations ER diagram (ERDPlus)

2. Normalized relational schema

Each entity and each many-to-many / attributed relationship in the diagram maps to its own table. Weak entities (Membership, OfficerPosition, Attendance, RoomReservation, Expense) carry a composite key that includes their owner's key.

Faculty(FacultyID, Name, Department)	PK: FacultyID
Club(ClubID, ClubName, Description, FoundedDate, AdvisorID)	PK: ClubID FK: AdvisorID -> Faculty.FacultyID
Student(StudentID, Name, Email, Major)	PK: StudentID
Membership(StudentID, ClubID, Role, JoinDate)	PK: (StudentID, ClubID) FK: StudentID -> Student FK: ClubID -> Club
OfficerPosition(StudentID, ClubID, PositionTitle, TermStart, TermEnd)	PK: (StudentID, ClubID) FK: (StudentID, ClubID) -> Membership
Event(EventID, EventName, Description, DateTime, ClubID)	PK: EventID FK: ClubID -> Club
Attendance(StudentID, EventID, Status, CheckInTime)	PK: (StudentID, EventID) FK: StudentID -> Student FK: EventID -> Event
Room(RoomNumber, Building, Capacity)	PK: (RoomNumber, Building)
RoomReservation(EventID, RoomNumber, Building, ReservationDate, StartTime, EndTime)	PK: EventID FK: EventID -> Event FK: (RoomNumber, Building) -> Room

Expense(ClubID, ExpenseID, Date, Description, Amount)	PK: (ClubID, ExpenseID) FK: ClubID -> Club
---	---

3. Design decision with multiple valid options

Decision: model OfficerPosition as a separate (weak) entity rather than as plain attributes on Membership.

Option A (rejected): add PositionTitle, TermStart, and TermEnd directly as attributes of Membership, since every officer is, by definition, also a member.

Option B (chosen): keep OfficerPosition as its own weak entity tied to Membership via the Holds relationship.

Reasoning: a membership spans the entire time a student belongs to a club (from JoinDate onward, no natural end date in the model), while an officer term has its own bounded TermStart/TermEnd and a student can serve multiple, non-contiguous terms (e.g., Treasurer one year, President the next) without ever leaving the club. Folding officer attributes into Membership would either force one row per membership (losing the history of multiple terms) or force artificial duplicate Membership rows just to represent different terms, both of which reintroduce redundancy and update anomalies. A separate weak entity keeps Membership representing 'is a member of' and lets OfficerPosition independently represent 'held this office during this term', which is both more accurate to the real-world rules and easier to query and extend (e.g., adding a new officer role later requires no schema change).

4. Example queries (in English)

1. Find all students who are officers in the Computer Science Club (join Student → Membership → OfficerPosition → Club, filtering on ClubName = 'Computer Science Club').
2. List all events scheduled for next week along with their room reservations (join Event → RoomReservation → Room, filtering on DateTime within the next 7 days).
3. Show the total expenses incurred by each club so far this year, sorted from highest to lowest total (group Expense by ClubID, sum Amount, filter on Date within the current year, join to Club for the display name).